

STACK

자료구조 스터디 Week3

학생회

 Made with Gamma

INDEX

01

THEORY

02

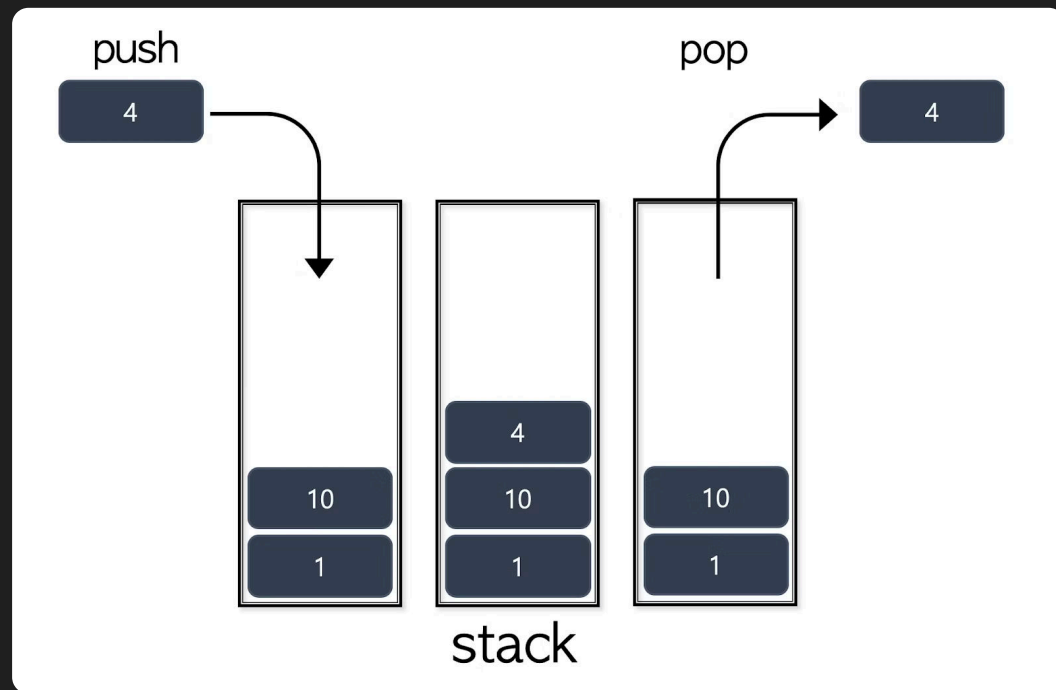
IMPLEMENTATION

03

PROBLEM SOLVING

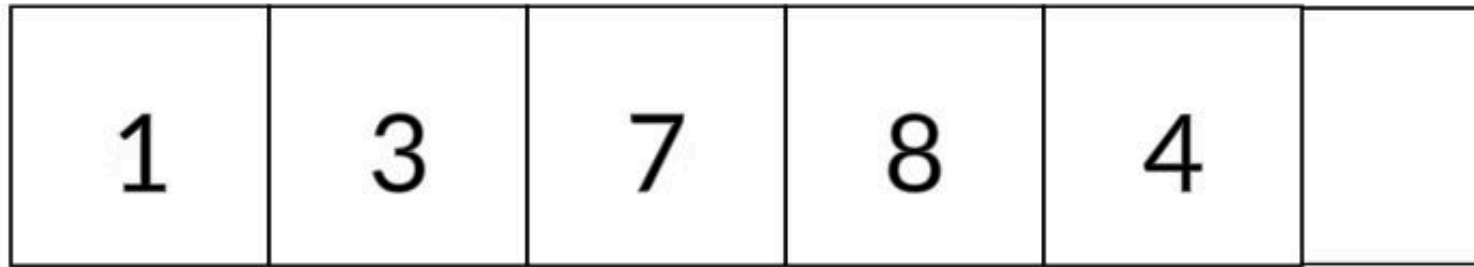
스택

스택(Stack)은 후입선출(LIFO, Last In First Out)의 자료구조로, 데이터를 쌓아 올린 형태로 저장하는 구조를 가지고 있습니다.



스택의 이해 - 삽입

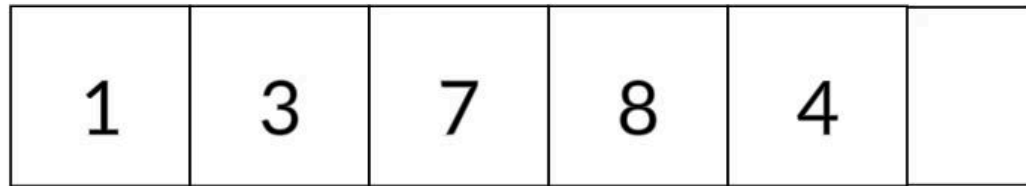
Top: 스택의 마지막 요소



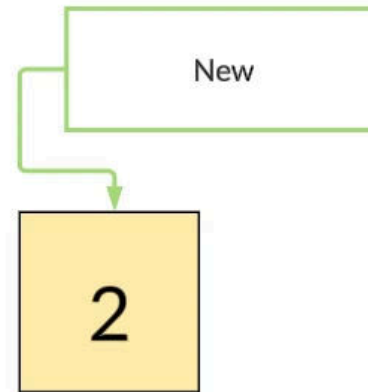
Top

스택의 이해 - 삽입

새로운 요소 2

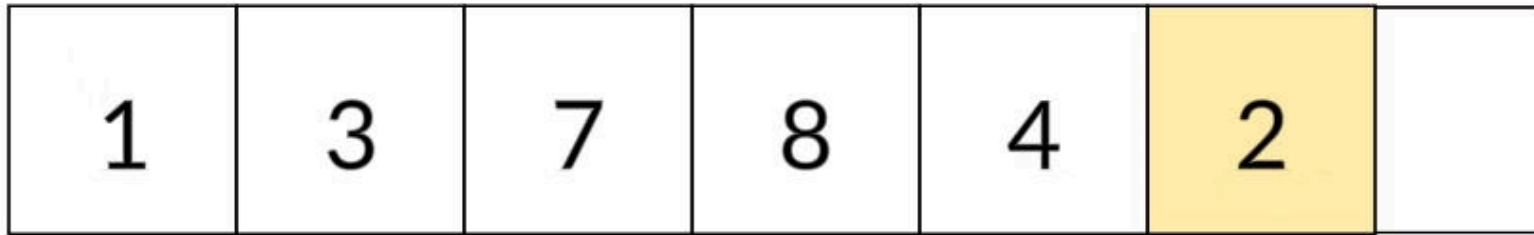


Top



스택의 이해 - 삽입

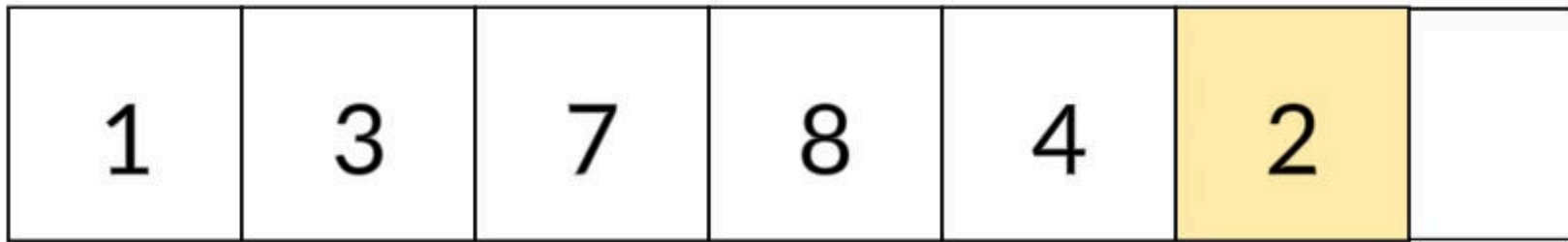
새로운 요소 2 Top의 뒤에 추가



Top

스택의 이해 - 삽입

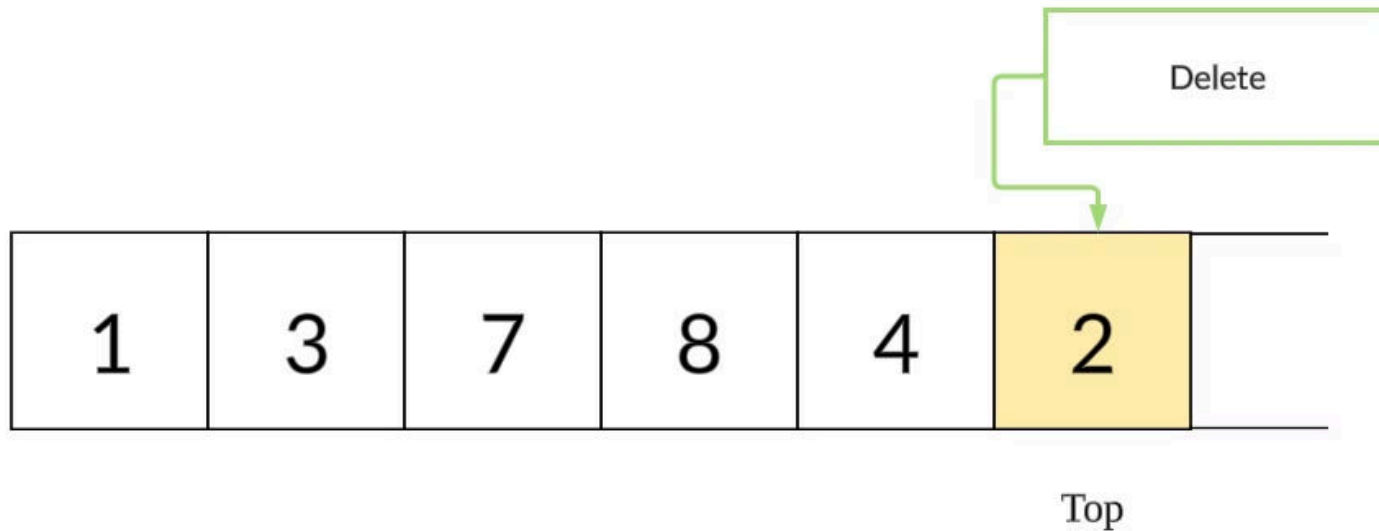
추가 이후 스택의 마지막 요소인 2가 Top



Top

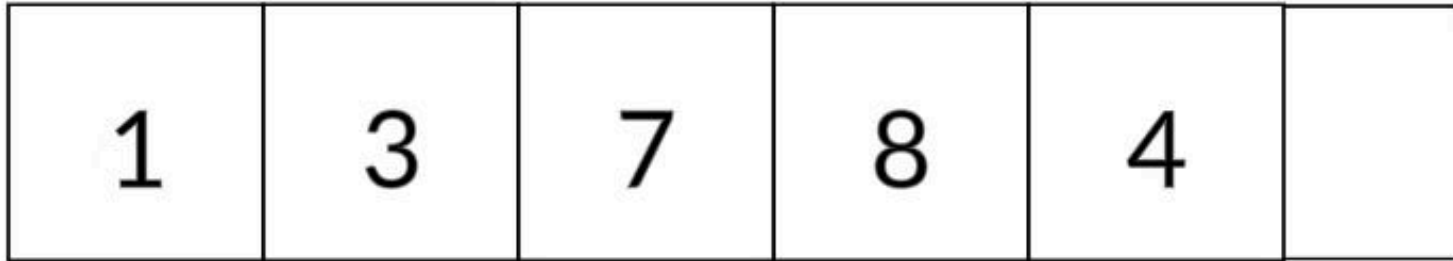
스택의 이해 - 삭제

스택의 Top(2) 제거



스택의 이해 - 삭제

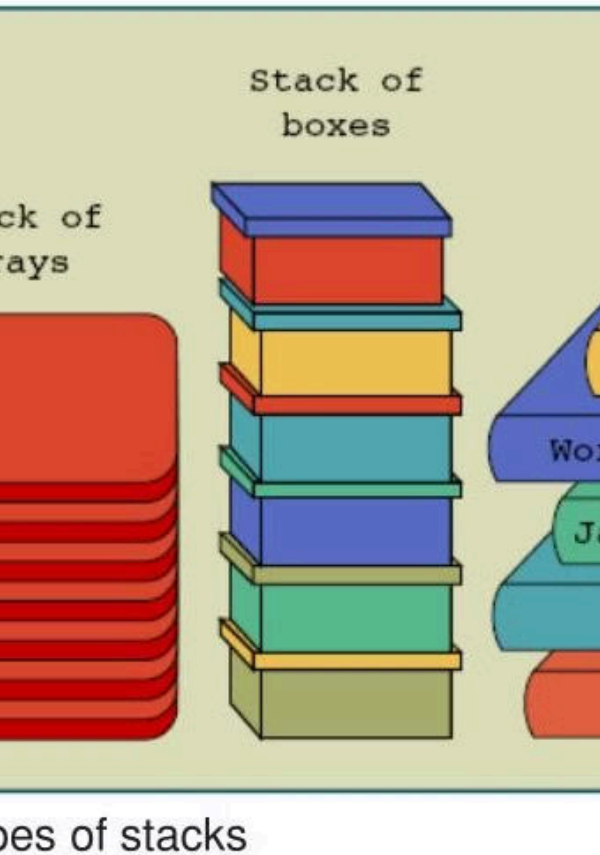
제거 이후 스택의 마지막 요소인 4가 Top



Top

IMPLEMENTATION

Stacks



스택의 구현 방법

1

배열

배열 기반 스택

2

링크드 리스트

링크드 리스트 기반 스택

스택의 기본 연산

1 isEmpty

스택이 비어 있는지 확인

2 Size

스택의 크기 조회

3 Top

가장 최근에 추가된 요소를 조회

4 Push

새로운 요소를 스택에 추가

5 Pop

가장 최근에 추가된 요소를 제거

ARRAY_STACK

배열 기반 스택 - 생성자

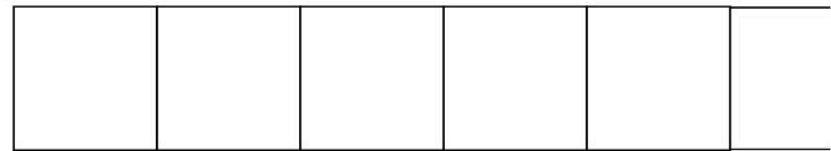
```
class Stack {  
private:  
    int arr[20]{};  
    int tos;  
public:  
    Stack() {  
        tos = 0;  
    }  
}
```

- 배열 선언
- tos는 top의 다음 인덱스

배열 기반 스택 - isEmpty

```
bool empty() {  
    return (tos == 0);  
}
```

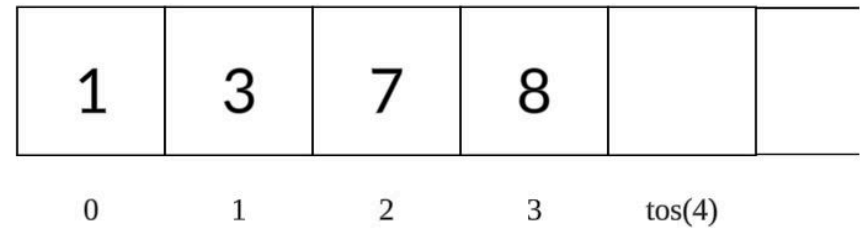
- tos가 0이라면 스택은 비어 있음



배열 기반 스택 - size

```
int size_() {  
    return tos;  
}
```

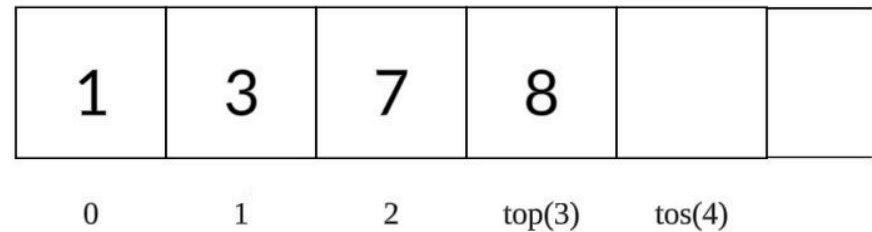
- tos는 size와 같은 값임



배열 기반 스택 - Top

```
void top() {  
    if (empty()) {  
        cout << "-1\n";  
        return;  
    }  
  
    cout << arr[tos - 1] << '\n';  
}
```

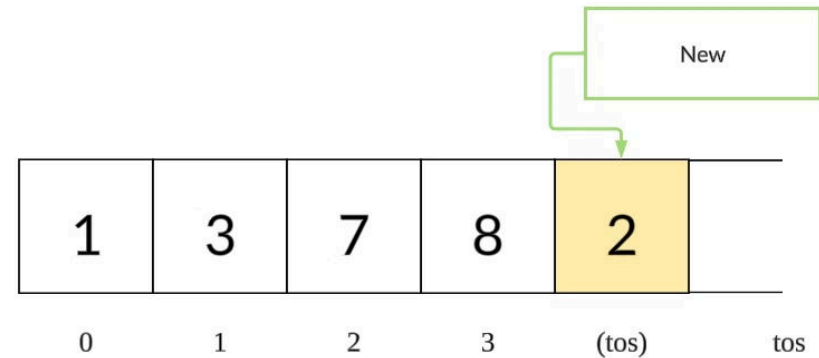
- top은 tos의 직전 요소



배열 기반 스택 - Push

```
void push(int x) {  
    if (full()) {  
        cout << "FULL\n";  
        return;  
    }  
  
    arr[tos++] = x;  
}
```

- 문제에 제시된 예외처리
- tos에 새로운 요소 추가, tos 업데이트



배열 기반 스택 - Pop

```
void pop() {  
    if (empty()) {  
        cout << "-1\n";  
        return;  
    }  
  
    top();  
    tos--;  
}
```

- 문제에 제시된 예외처리
- top 출력, tos 업데이트

LIST_STACK

링크드 리스트 기반 스택 - 생성자

```
class Stack {  
private:  
    LinkedList ll;  
    int size;  
  
public:  
    Stack() {  
        ll = LinkedList();  
        size = 0;  
    }  
}
```

- 링리 선언
- size = tos라 생각!

링크드 리스트 기반 스택 - isEmpty

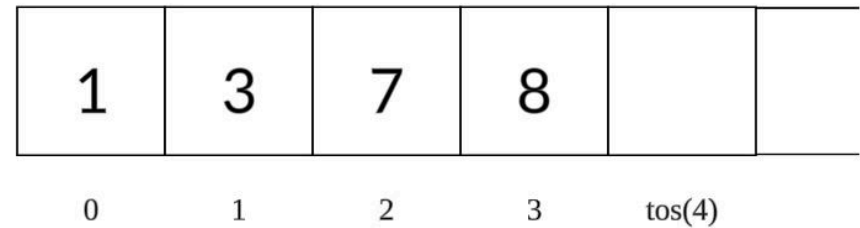
```
bool empty() {  
    return (size == 0);  
}
```

- tos가 0이라면 스택은 비어 있음

링크드 리스트 기반 스택 - Size

```
int size_() {  
    return size;  
}
```

- tos는 size와 같은 값임



링크드 리스트 기반 스택 - Top

```
void top() {  
    if (empty()) {  
        cout << "-1\n";  
    }  
  
    else {  
        cout << ll.tail->value << "\n";  
    }  
}
```

- 문제에 제시된 예외처리
- 링리의 tail이 스택의 top

링크드 리스트 기반 스택 - Push

```
void push(int x) {  
    ll.append(x);  
    size++;  
}
```

- 문제에 제시된 예외처리
- 링리의 append가 스택의 push
- size 1 증가

링크드 리스트 기반 스택 - Pop

```
void pop() {  
    if (empty()) {  
        cout << "-1\n";  
    }  
  
    else {  
        cout << ll.delete_back() << "\n";  
        size--;  
    }  
}
```

- 문제에 제시된 예외처리
- 링리의 delete_back이 스택의 pop
- size 1 감소

PROBLEM SOLVING

**THANK'S FOR
WATCHING**